

01
Split
Paste → documents
splitText()

02
Embed
64-d vector / doc
embedOllama()

03
Extract
5-type NER + relations
extractLLM()

04
kNN
Search from new docs
knnQuery()

05
Graph
IDW weights, θ gate
computeEdges()

06
Send
Nodes · edges · graph
graphScript()

ONE SENTENCE → TWO KINDS OF EDGE

1 RELATION stated in text · asserted

DOC 12
NEW
John Doe works for Company ABC.

→
LLM
relation

Company ABC
BUSINESS

WORKS weight 1.0

John Doe
PEOPLE

2 EMBEDDED inferred by document similarity

DOC 12
DOC 5
INDEXED

→
kNN
+ IDW

John Doe
PEOPLE

EMBEDDED weight 0.81

Company ZZZ
BUSINESS

EMBEDDED EDGE WEIGHT · TRUE-IDW

// over shared doc pairs $a \in A, b \in B$
 $\cos = v_a \cdot v_b \quad d = 1 - \cos$
 $w = 1 / (\epsilon + d) \quad v = (1 + \cos) / 2$
 $\text{weight} = \Sigma(w \cdot v) / \Sigma(w) \in (0, 1]$

Relation edges (WORKS, FOUNDED, LOCATED-IN) skip this – asserted, so **weight = 1.0**.
Both layers gate at **$\theta = 0.5$** .

FIVE-TYPE ONTOLOGY

● People

● Organization

● Business

● Facility

● Location

Block-key merge folds surface variants into stable canonical nodes; new variants become aliases, the PK never renames.

WHY IT SCALES

Each **new** document runs one TOP-k kNN over the HNSW-indexed table. Old ↔ old pairs are never revisited — their IDW lives in running $\Sigma(w \cdot v)$ and $\Sigma(w)$ accumulators.

~~$O(\text{all}^2)$~~ → $O(\text{new} \cdot k)$

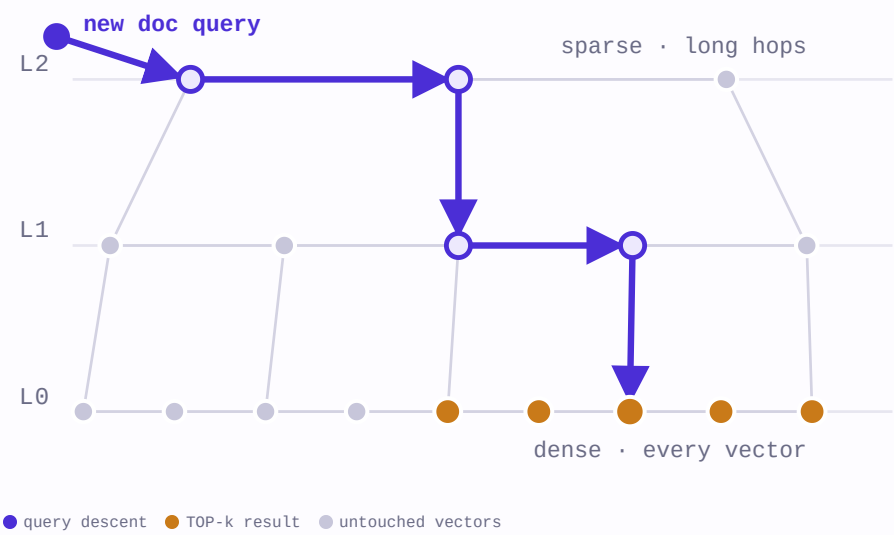
Kinetica output: graph_nodes graph_edges graph_membership → CREATE UNDIRECTED GRAPH, cost = (1 - weight)

Append mode: doc_id = MAX+1

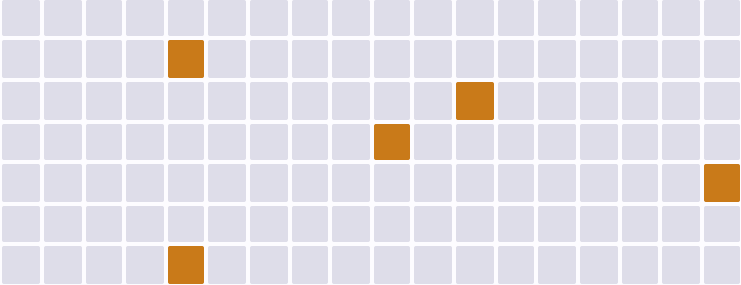
Search only from new documents

WHAT IS HNSW

Hierarchical Navigable Small World — an approximate nearest-neighbor index built as a stack of proximity graphs. The top layer is sparse with long-range links for big jumps; each layer down is denser; the base holds every vector. A query greedily hops toward its neighbors, descending layer by layer, so a lookup costs $\sim O(\log N)$ — not a full $O(N)$ scan. It updates automatically as rows are inserted, which is why it is the default for an append-target table.



TOP-K KNN · ONE QUERY PER NEW DOC (K = 5, MADE UP)



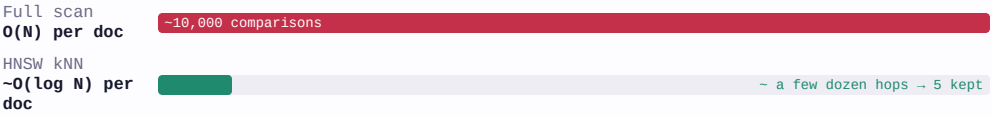
126 of $\sim 10,000$ indexed vectors shown · HNSW returns the 5 nearest · the rest are never compared.

#	doc	sim
1	D-0512	0.62
2	D-0088	0.55
3	D-0041	0.48
4	D-0007	0.44
5	D-0973	0.41

cutoff β : distance ≤ 0.60

THE QUERY & WHY IT STAYS CHEAP

```
-- per new doc: bounded by the index, not the table size
SELECT TOP 5 doc_id,
      1 - COSINE_DISTANCE(embedding, :q) AS sim
FROM   vector_embeddings
WHERE  COSINE_DISTANCE(embedding, :q) <= 0.60
ORDER BY COSINE_DISTANCE(embedding, :q);
```



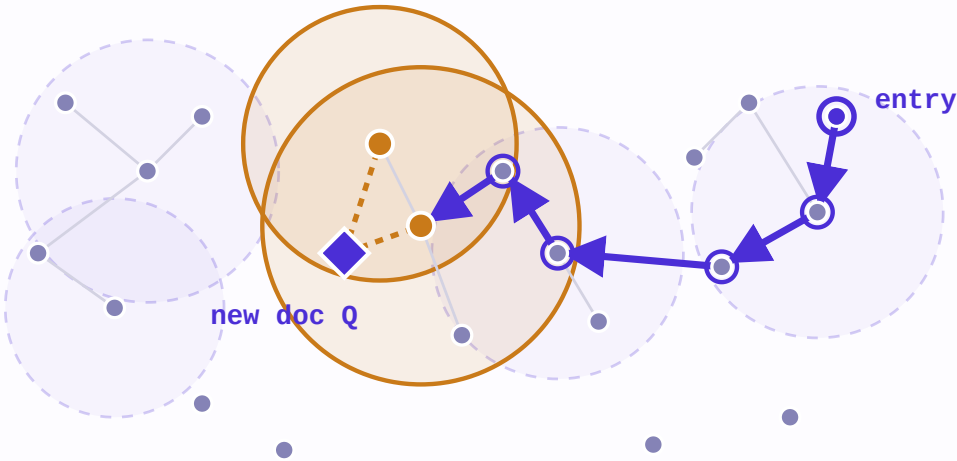
Only new docs are query origins. Old-old pairs already live in the `graph_edges` accumulators – total cost $O(\text{new} \cdot \log N)$, never $O(N^2)$.

k, β · tunable · ADD HNSW INDEX

Documents are points, neighbors are reach

INTUITION · vector space
k-NN = who's in my ring
each dot = one document

THE EMBEDDING SPACE, FLATTENED TO A PLANE



position $\approx$ similarity · ring = the k nearest neighbors

● new document Q — greedy walk from entry ● neighbors it lands on ● other documents

READING THE PICTURE

- Each dot is a **document**, placed by its embedding — closer dots are more similar. 64 dimensions, flattened to 2D here.
- The ring around a dot covers its **N nearest neighbors** — its reach. Every dot has one; a few are drawn to keep it legible.
- The thin links are the stored neighbor edges — this is the HNSW base layer.
- The search starts at an **entry point** on the far side and walks neighbor → neighbor, each hop landing closer, until it steps onto Q's disk — it never scans every dot.

WHEN A NEW DOCUMENT ARRIVES

1. Embed it → it drops in as point **Q**.
 2. Find which rings Q falls inside — its nearest neighbors. HNSW navigates there in $\sim O(\log N)$, without testing all N rings (slide 2).
 3. Those become Q's **index links** and the source docs whose entities earn an **EMBEDDED** edge to Q's.
- The closer Q sits, the smaller the distance $d = 1 - \cos$ and the larger the edge weight $(1 + \cos) / 2$.

The ring is the intuition; HNSW is the machinery — it finds the landing neighborhood without checking all N rings.

Q's neighbors → entities → graph_edges